Validación en Contextos Reales o Simulados

SEMANA 5

Fase de Evaluación y Optimización · Maestría en Inteligencia Artificial · Proyecto Integrador





Objetivos de la Semana

- **Aplicar** evaluación estadística robusta y métricas del dominio
- **Diagnosticar** overfitting, underfitting y problemas de generalización
- **Detectar y mitigar** sesgo en modelos de IA
- **Optimizar** hiperparámetros y analizar su sensibilidad
- **Validar** modelos en contextos reales o simulados

Mapa de la Semana

1

Presentación de la fase de Evaluación y Optimización

Métricas, estadística robusta, diagnóstico, sesgo, generalización e hiperparámetros.

2

Workshop: Simulación de escenarios críticos

Pruebas de estrés al modelo con ruido, datos faltantes, desbalance y cambios de distribución.

3

Workshop: Análisis de sensibilidad de hiperparámetros

Partial dependence plots, importancia de hiperparámetros y efectos de interacción.

Del Laboratorio al Mundo Real

En el laboratorio

- Datos curados y etiquetados
- Partición aleatoria train/test
- Métricas agregadas en un solo número
- Los errores no tienen costo

En el mundo real

- Ruido, datos faltantes y drift
- La distribución cambia en el tiempo y el espacio
- Errores con costo asimétrico
- Restricciones de latencia y recursos

Principio Fundamental

Un buen resultado en el conjunto de prueba **no garantiza** buen desempeño en producción. Validar es probar el modelo bajo las condiciones donde será usado.

Esta semana: cómo evaluar con rigor, optimizar con criterio y validar en contextos reales o simulados.



Implementación de Métricas Básicas

Checklist inicial

- Definir la métrica objetivo según el costo del error
- Establecer un baseline simple (modelo dummy)
- Reportar varias métricas complementarias
- Separar train, validación y test

```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import classification_report, roc_auc_score

base = DummyClassifier(strategy="stratified").fit(X_train,
y_train)

def evaluar(modelo, X, y):
    y_pred = modelo.predict(X)
    y_prob = modelo.predict_proba(X)[:, 1]
    print(classification_report(y, y_pred))
    return {"auc": roc_auc_score(y, y_prob)}

evaluar(base, X_val, y_val)  # el modelo debe superar este piso
```

Evaluación Estadística Robusta

Validación cruzada

- Stratified k-fold repetido
- GroupKFold si hay grupos (sitios, sujetos)
- TimeSeriesSplit en series temporales

Incertidumbre

- Media ± desviación estándar
- Intervalos de confianza por bootstrap
- Semillas aleatorias múltiples

Pruebas de significancia

- McNemar para dos clasificadores
- Wilcoxon sobre folds pareados
- t pareado corregido

Regla práctica

Una diferencia de 0.5% entre modelos **no es una mejora** si cae dentro del intervalo de confianza. Reporta la variabilidad, no solo el promedio.



Herramientas: CV Repetida e IC

```
import numpy as np
from sklearn.model_selection import (
    RepeatedStratifiedKFold, cross_val_score)

cv = RepeatedStratifiedKFold(
    n_splits=5, n_repeats=3, random_state=42)
scores = cross_val_score(
    modelo, X, y, cv=cv, scoring="f1_macro")
print(f"F1 = {scores.mean():.3f} ± {scores.std():.3f}")

# IC 95% por bootstrap
rng = np.random.default_rng(42)
boot = [rng.choice(scores, len(scores)).mean()
        for _ in range(2000)]
bajo, alto = np.percentile(boot, [2.5, 97.5])
```

Qué observar

- 15 evaluaciones en lugar de una sola
- Un IC amplio indica un modelo inestable
- Si los IC de dos modelos se solapan, no hay evidencia de mejora

Métricas Específicas del Dominio

Clasificación general
Exactitud, F1 macro, AUC-ROC y AUC-PR

Visión por computador
IoU, Dice y mAP

Lenguaje natural
BLEU, ROUGE, BERTScore y perplexity

Series temporales
MASE, sMAPE y pinball loss para cuantiles

Salud y diagnóstico
Sensibilidad a especificidad fija, AUC-PR y calibración

Negocio y riesgo
Costo esperado, tasa de falsas alarmas y lift

Costo esperado: FP × cFP + FN × cFN. Elige el umbral que minimice el costo, no el que maximice la exactitud.



Diagnóstico de Overfitting y Underfitting

Underfitting

Señal: error alto en train y en validación.

Acción: más capacidad, mejores features, menos regularización.

Buen ajuste

Señal: error bajo y similar en train y validación.

Acción: confirmar en test y en datos nuevos.

Overfitting

Señal: error bajo en train y alto en validación.

Acción: regularización, más datos, early stopping, dropout.

Herramientas de diagnóstico

Curvas de aprendizaje (error vs. tamaño de datos), curvas de validación (error vs. hiperparámetro) y curvas de pérdida por época en redes neuronales.



Herramientas: Curvas de Aprendizaje

```
import numpy as np
from sklearn.model_selection import learning_curve

sizes, train, val = learning_curve(
    modelo, X, y, cv=5, scoring="f1_macro",
    train_sizes=np.linspace(0.1, 1.0, 8))

brecha = train.mean(axis=1) - val.mean(axis=1)
print("Brecha train-val:", brecha.round(3))

# Overfitting si la brecha es grande y no se cierra
# Underfitting si ambas curvas son bajas y convergen
```

Cómo leerla

- Brecha grande: overfitting
- Ambas bajas y juntas: underfitting
- Curvas que convergen: más datos no ayudan, cambia el modelo
- Val sube con más datos: conviene recolectar más

Detección y Mitigación de Sesgo

Detectar

- Fuentes: muestreo, etiquetado y medición
- Evaluar las métricas por subgrupo, no solo en promedio
- Paridad demográfica: misma tasa de predicción positiva
- Igualdad de oportunidad: mismo recall entre grupos
- Equalized odds: mismo TPR y FPR entre grupos

Mitigar

- **Pre-procesamiento:** reponderar o re-muestrear los datos
- **In-procesamiento:** restricciones de equidad durante el entrenamiento
- **Post-procesamiento:** umbrales de decisión por grupo

Ojo con los subgrupos poco representados

El sesgo aparece cuando ciertos grupos (género, edad, región, tipo de dispositivo) casi no están en los datos. Mejorar un grupo puede empeorar otro: documenta el compromiso.

Herramientas: Fairlearn

```
from fairlearn.metrics import MetricFrame, selection_rate
from sklearn.metrics import recall_score

mf = MetricFrame(
    metrics={"recall": recall_score,
             "sel_rate": selection_rate},
    y_true=y_test, y_pred=y_pred,
    sensitive_features=grupo_test)

print(mf.by_group)
print("Diferencia máxima:", mf.difference())
```

Interpretación

- by_group muestra cada métrica por subgrupo
- difference() mide la mayor brecha entre grupos
- Define un umbral de brecha aceptable antes de evaluar
- Repite el análisis después de mitigar

Problemas de Generalización

Data leakage

Información del test que se filtra al entrenamiento: escalar antes de dividir, duplicados, ventanas temporales solapadas.

Covariate shift

Cambia la distribución de las entradas: otro dispositivo, otra población u otra temporada.

Concept drift

Cambia la relación entre entradas y etiqueta con el tiempo, aunque las entradas parezcan iguales.

Autocorrelación

Vecinos en espacio o tiempo se parecen, y un split aleatorio infla las métricas.

Contramedidas: pipelines que ajustan solo con train, GroupKFold por usuario o sitio, TimeSeriesSplit y monitoreo de drift.

Optimización de Hiperparámetros

Método	Idea	Cuándo usarlo
Grid Search	Prueba todas las combinaciones de una malla	Pocos parámetros y rangos pequeños
Random Search	Muestrea combinaciones al azar	Muchos parámetros, presupuesto limitado
Bayesiana (Optuna)	Aprende de las pruebas previas	Entrenamientos costosos
Hyperband	Descarta pronto las configuraciones malas	Redes profundas y presupuesto de cómputo

Regla de oro: optimiza con validación cruzada sobre train y reserva el test para una evaluación final única.

Herramientas: Optuna

```
import optuna
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

def objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 50, 500),
        "max_depth": trial.suggest_int("max_depth", 3, 20),
        "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1,
20),
    }
    modelo = RandomForestClassifier(**params, random_state=42)
    cv = cross_val_score(modelo, X_train, y_train, cv=5,
                         scoring="f1_macro")
    return cv.mean()

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=50)
```

Buenas prácticas

- Escala logarítmica para tasas
- Semilla fija y registro de trials
- Usa pruning si el entrenamiento es largo
- Test solo al final

Validación en Contextos Reales

1. Offline
Hold-out y validación cruzada

→

2. Simulación
Escenarios críticos sintéticos

→

3. Sombra
Predice en vivo sin afectar decisiones

→

4. Piloto
Canary o prueba A/B con tráfico limitado

→

5. Monitoreo
Drift, desempeño y alertas

Criterio de avance

Cada etapa define umbrales de éxito y una condición de retroceso (rollback). Sin umbrales definidos de antemano, no hay validación.

Simulado vs. real: si no hay acceso a producción, simula con datos de otra población, época o fuente.

Workshop: Simulación de Escenarios Críticos

Escenario	Cómo simularlo	Qué medir
Ruido en la entrada	Ruido gaussiano con σ creciente	Caída de F1
Datos faltantes	Enmascarar 10% a 50% de las features	Degradación por feature
Desbalance extremo	Reducir la clase minoritaria a 1% a 5%	Recall y AUC-PR
Cambio de distribución	Entrenar con un subgrupo y probar con otro	AUC y calibración
Feature clave caída	Una variable importante llega en cero o constante	Recall y F1 por clase

Pregunta guía: ¿hasta qué nivel de perturbación el modelo sigue siendo útil, y qué haría el sistema cuando deje de serlo?

Workshop 1: Prueba de Estrés en Código

```
import numpy as np
from sklearn.metrics import f1_score

def prueba_estres(modelo, X, y, sigmas=(0, 0.1, 0.2, 0.5)):
    rng = np.random.default_rng(42)
    resultados = {}
    for s in sigmas:
        X_ruido = X + rng.normal(0, s, X.shape)
        y_pred = modelo.predict(X_ruido)
        resultados[s] = f1_score(y, y_pred, average="macro")
    return resultados

# Graficar sigma vs. F1 y marcar el umbral aceptable
```

Qué entregar

- Tabla de degradación por escenario
- Umbral de degradación aceptable, definido antes de correr
- Plan de mitigación para los escenarios que fallan

Workshop: Sensibilidad de Hiperparámetros

Partial dependence plots

Efecto individual de cada parámetro sobre la métrica, promediando los demás.

Feature importance for hyperparameters

Ranking de importancia: qué parámetros explican la variación del desempeño.

Interaction effects

Análisis de interacciones entre parámetros, por ejemplo profundidad × número de árboles.

Pregunta guía

¿Qué hiperparámetros merecen presupuesto de búsqueda y cuáles se pueden fijar sin perder desempeño?

Insumo: las pruebas (trials) del estudio de Optuna que ya ejecutaste en la actividad de optimización.

Workshop 2: Sensibilidad en Código

```
from optuna.visualization import (
    plot_param_importances, plot_slice, plot_contour)

plot_param_importances(study).show()  # ranking
plot_slice(study).show()  # efecto individual
plot_contour(study, params=["max_depth", "n_estimators"]).show()

# PDP sobre las pruebas, con un modelo sustituto
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import PartialDependenceDisplay
cols = ["params_max_depth", "params_n_estimators"]
df = study.trials_dataframe()
df[cols] = df[cols].astype(float)
sustituto = RandomForestRegressor(random_state=42)
sustituto.fit(df[cols], df["value"])
PartialDependenceDisplay.from_estimator(sustituto, df[cols], [0,
1])
```

Cómo interpretar

- Curva plana: parámetro poco sensible, se puede fijar
- Pendiente fuerte: amplía el rango de búsqueda
- Valle diagonal en el contorno: hay interacción entre parámetros

Ejemplo de Workshop: el Caso

Clasificar tumores

- **Datos:** Breast Cancer Wisconsin, incluido en scikit-learn
- 569 muestras, 30 variables, 37% malignos
- **Modelo:** Random Forest con baseline dummy
- **Herramientas:** scikit-learn y Optuna

Por qué este caso

- Un falso negativo cuesta más que un falso positivo
- La métrica clave es el **recall** de la clase maligna
- No requiere experiencia en ningún dominio
- Corre en pocos minutos en cualquier laptop

Parte 0 · Baseline y evaluación con IC · 20 min

Workshop 1 · Escenarios críticos · 60 min

Workshop 2 · Sensibilidad · 70 min

Duraciones sugeridas; ajústalas a tu proyecto. Notebook de apoyo: Semana5_Workshops.ipynb

Ejemplo Workshop 1: Qué Verán los Estudiantes

Escenario	Resultado (semilla 42)	Lección
Ruido σ=1.0	Precisión de 1.00 a 0.81; recall estable en 0.92	El ruido genera falsas alarmas
Faltantes 50%	Recall de 0.92 a 0.58	Los faltantes causan casos perdidos
Desbalance 1%	Accuracy sube a 0.999; recall sigue en 0.92	La accuracy engaña
Otro subgrupo	Recall de 1.00 (mismo subgrupo) a 0.84	El desempeño no siempre se transfiere
3 variables caídas	Recall de 0.92 a 0.80	Dependencia de pocas variables

Discusión: con una caída máxima aceptable de 5 puntos de recall, fallan los faltantes (30% y 50%), el cambio de subgrupo y la caída de worst area o de las 3 variables juntas. ¿Qué mitigación propone cada grupo?

Ejemplo Workshop 2: Qué Verán los Estudiantes

Importancia

min_samples_leaf concentra cerca del 80%. n_estimators y max_features pesan menos del 3% cada uno.

Partial dependence

El F1 baja al aumentar min_samples_leaf. Las curvas de max_depth y max_features son casi planas.

Magnitud del efecto

Las 60 pruebas quedan entre 0.90 y 0.93 de F1: toda la búsqueda mueve el resultado muy poco.

Decisión

Fijar n_estimators y max_features, y buscar solo min_samples_leaf y max_depth.

Punto clave: en test, el modelo optimizado (F1 0.942) no supera al baseline (F1 0.959). La diferencia cae dentro de la variabilidad de la parte 0, así que optimizar tiene retornos decrecientes aquí.

Presentación de la Fase de Evaluación y Optimización

1. **Métricas básicas**
2. **Evaluación estadística robusta**
3. **Métricas del dominio**
4. **Diagnóstico de overfitting y underfitting**
5. **Detección y mitigación de sesgo**
6. **Problemas de generalización**
7. **Optimización de hiperparámetros**
8. **Validación en contextos reales**

En cada sección presenta tres cosas: la **decisión** tomada, la **evidencia** que la respalda y las **limitaciones** que reconoces.

Mejores Prácticas en Evaluación

Checklist de evaluación

- Definir métricas y umbrales de éxito antes de entrenar
- Establecer baselines y modelos dummy
- Evaluar con múltiples splits y semillas
- Analizar los errores de forma cualitativa
- Registrar experimentos (MLflow o W&B)

Errores comunes a evitar

- **Data leakage:** información del test en el entrenamiento
- **Cherry picking:** reportar solo las mejores métricas
- **Overfitting al test:** ajustar decisiones mirando el test
- **Ignorar el costo del error** y el contexto de uso
- **No documentar** semillas, versiones ni decisiones



Conclusiones

- **Evaluar con rigor:** varias métricas, validación cruzada e intervalos de confianza
- **Diagnosticar** sobreajuste, sesgo y problemas de generalización
- **Optimizar** con criterio y analizar la sensibilidad de los hiperparámetros
- **Validar** bajo escenarios críticos antes de llegar a producción
- **Documentar** decisiones y limitaciones

